

Operating System: Unix

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Syllabus

- **Session 1:** Introduction to Unix/Linux
- **Session 2:** Installation of Linux (WSL, VirtualBox, Mac)
- **Session 3:** Linux Architecture & File Systems
- **Session 4:** CLI Basics: Commands & Navigation
- **Session 5:** Advanced CLI: I/O & Text Processing
- **Session 6:** Shell Scripting I: Basics & Automation
- **Session 7:** Shell Scripting II: Advanced Scripts
- **Session 8:** Processes & Init Systems
- **Session 9:** Users, Groups & Permissions
- **Session 10:** Text Editors: vi/vim
- **Session 11:** Github ? Networking?
- **Session 12:** Remote Access, Downloads & Shell Customization

Linux Architecture & File Systems

Session 3

Linux Architecture & File Systems

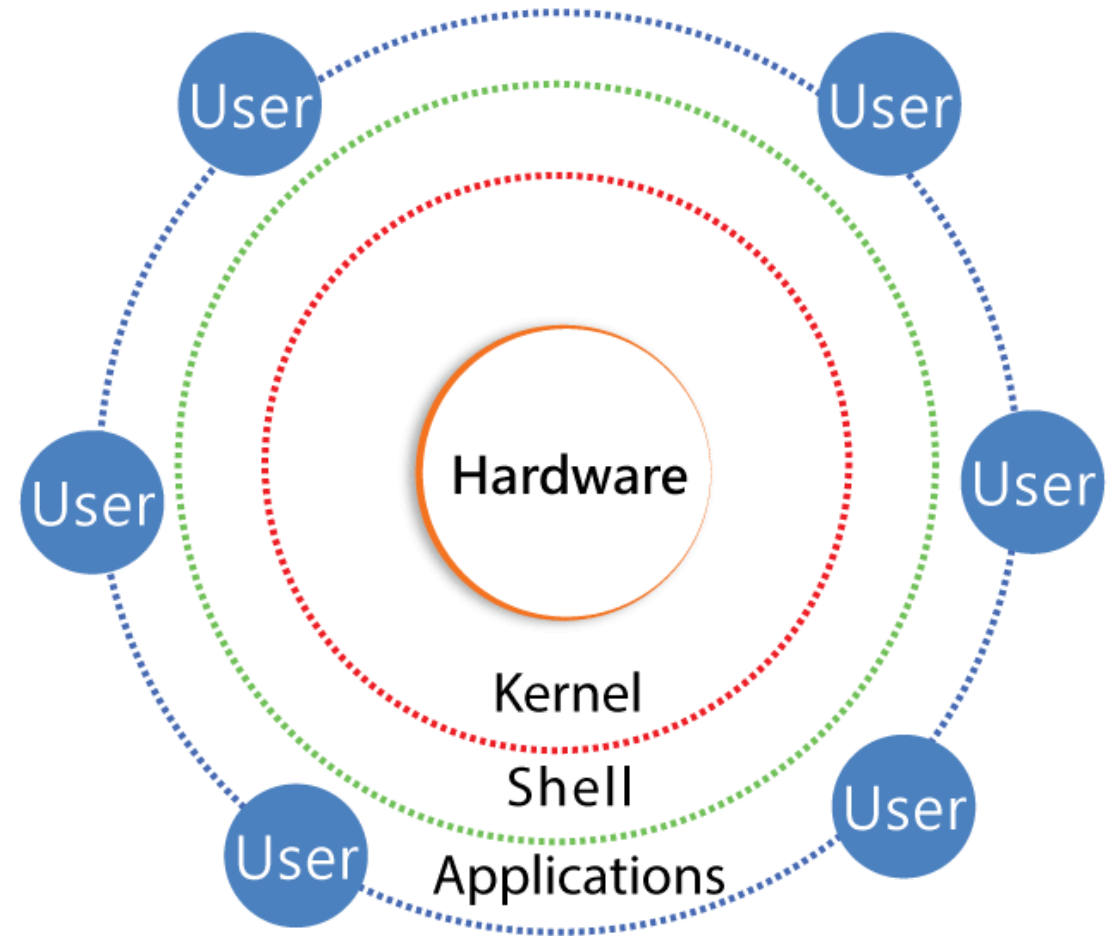
Session 3

Objectives:

- Identify Linux system components (kernel, shell, userland)
- Understand basic Linux boot process
- Explore key directories and their purposes
- Practice navigation and observation in Linux filesystem

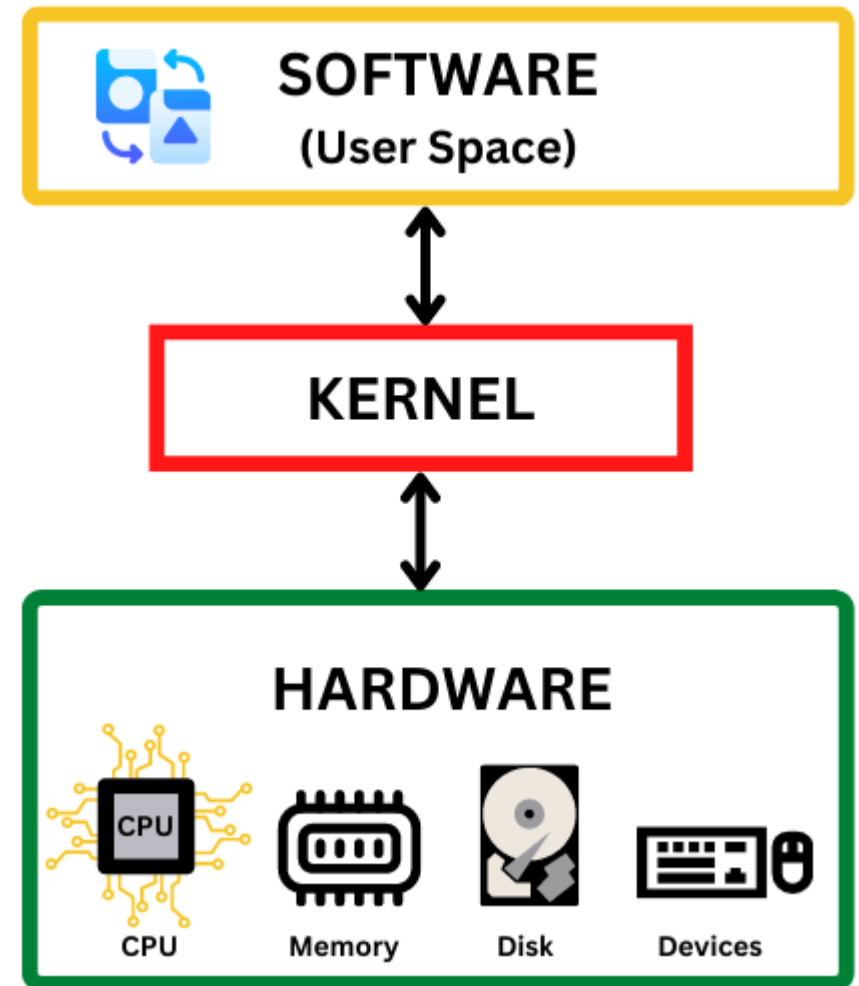
System structure

- **Hardware:**
- **Kernel:**
- **Shell:**
- **User Applications:**



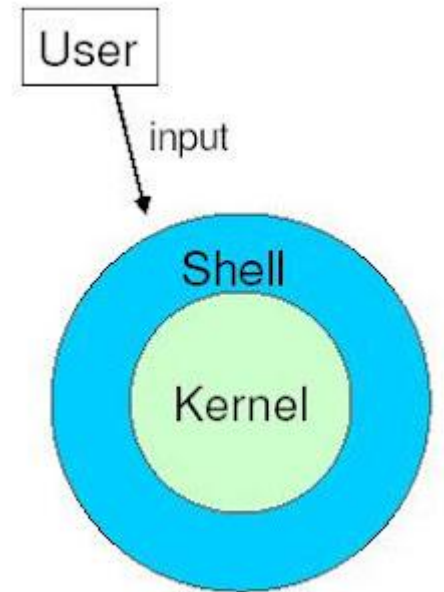
Kernel: core of Linux

- Operating system “kernel” is the core software used to “talk” to computer hardware
- It’s a core and modular system of drivers used to create a standardized environment for interfacing with hardware
- Resource manager for allocating memory and time to system and user processes as well as interacting with files (I/O)
- Kernel operates in its own memory or “kernel-space”



Shell: user interface

- On user log-in, the system runs a shell
- A shell is the environment within which you will interface with the kernel via commands
- It determines the syntax for complex command-line operations and shell scripting.
- The shell you're using is called “bash,” the successor to the venerable “Bourne Shell” called “sh”
- BASH: “Bourne Again SHell”



More Shell

- What shell am I in?

Typing “echo \$SHELL” will show you!

- You should see ‘/bin/bash’
- Typing “echo \$0” will also show your shell
- \$SHELL and \$0 are shell variables...more about variables later
- List of available shells on the system can be displayed by typing “chsh --list-shells”

The chsh command can be used to change your default shell as well

Shell Preference

- When you login, startup scripts are run to setup your environment
- For bash, you can customize your environment by adding or modifying environment variables and aliases in the **.bashrc** file in your home directory.
- Examples:
 - alias ls='ls -rtl' alias
 - bwulf='ssh \$USER@epita.fr'
 - PATH=\$PATH:/data/myusername
 - EDITOR=/usr/bin/vim

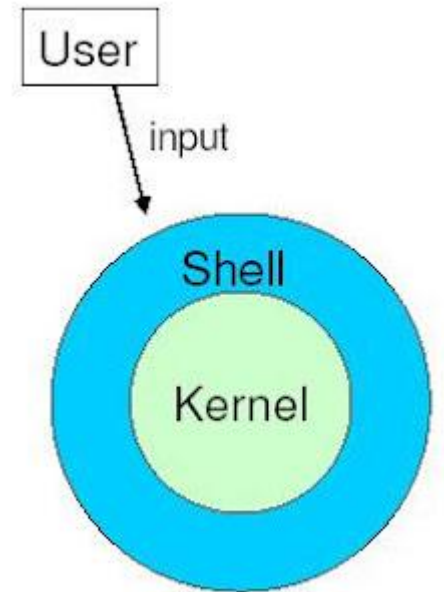
Different Shells

- sh – the original UNIX shell (Bourne shell)
- bash – written as a replacement/extension of sh
- csh – C shell based on the C programming language developed in the late 1970s
- tcsh – enhanced version of C shell
- ksh – Korn shell developed in the early 1980's, backward compatible with sh, but with some features of csh
- zsh – extended version of sh, developed in 1990
- dash – developed as replacement for ash in Debian



Applications

- Applications: browsers, text editors, games
- Services: web servers, mail servers
- Everything running on top of shell and kernel



Boot process

What really happens behind the scenes
When you press that power button to on!!

```
Configuring ISA PNP
Setting system time from the hardware clock (localtime).
Using /ect/random-seed to initialize /dev/random.
Initializing basic system settings ...
Updating shared libraries
Settings hostname: server23.kernel.org
INIT: Entering runlevel: 4
rc.M ==> Going multiuser...
Starting system logger ... [OK]
Initialising advanced hardware
Setting up modules ... [OK]
Initialising network
Setting up localhost ... [OK]
Setting up inet1 ... [OK]
Setting up route... [OK]
Setting up loca fancy console and GUI
Loading fc-cache ... [OK]
rc.vlinit ==> Going to runlevel 4
```

Viewing the Boot Process

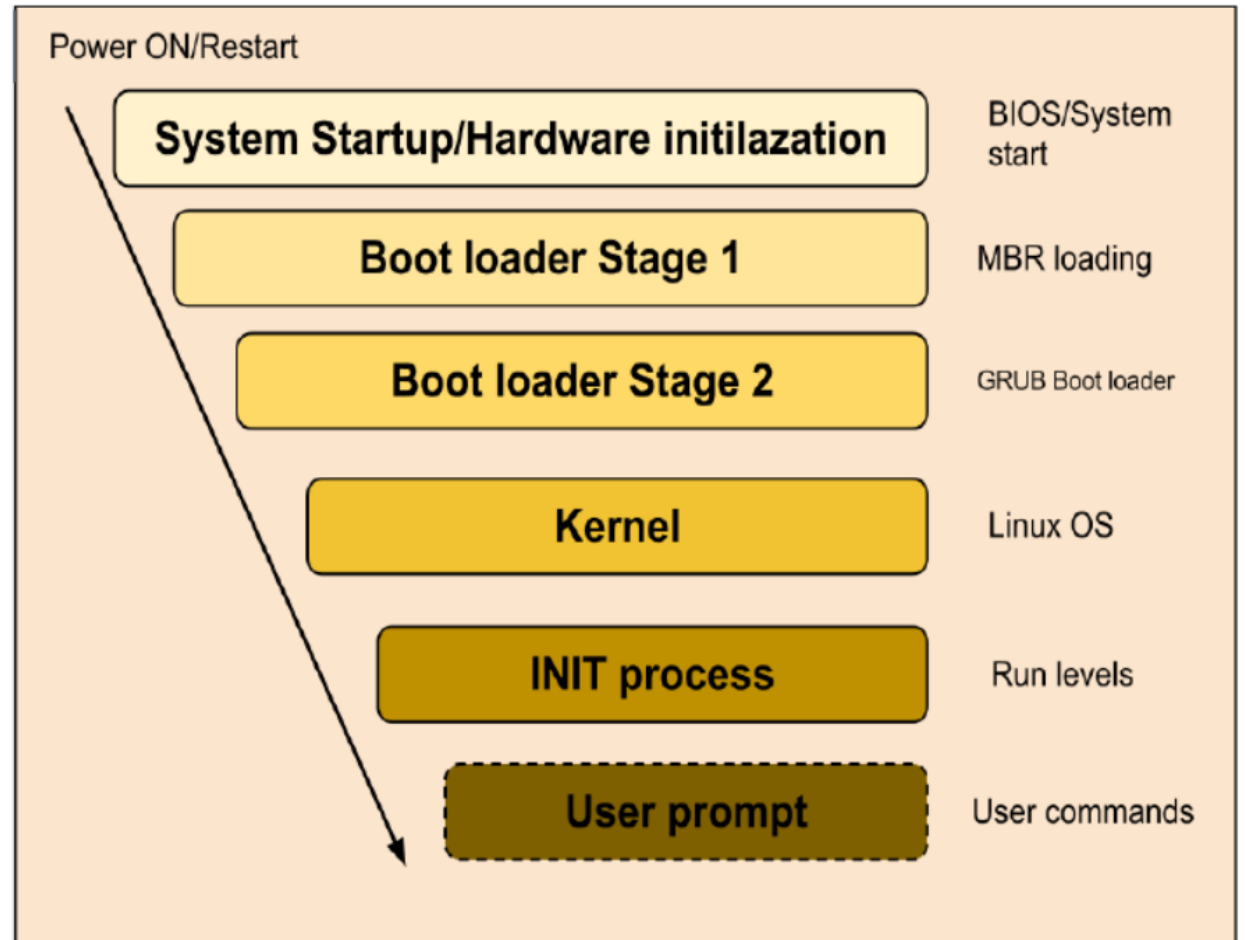
Boot kernel messages are stored into

- a buffer in memory, called the kernel ring buffer
- could be reviewed using the dmesg command; `sudo dmesg`
- a log file, usually in the `/var/log` folder
- `Sudo more /var/log/boot.log`

How Linux boots?

A quick view of booting sequence:

- Power on
- CPU jumps to BIOS
- BIOS runs POST
- Finds first bootable device
- Load and execute MBR
- Load OS
- User prompt



BIOS/UEFI Initialization

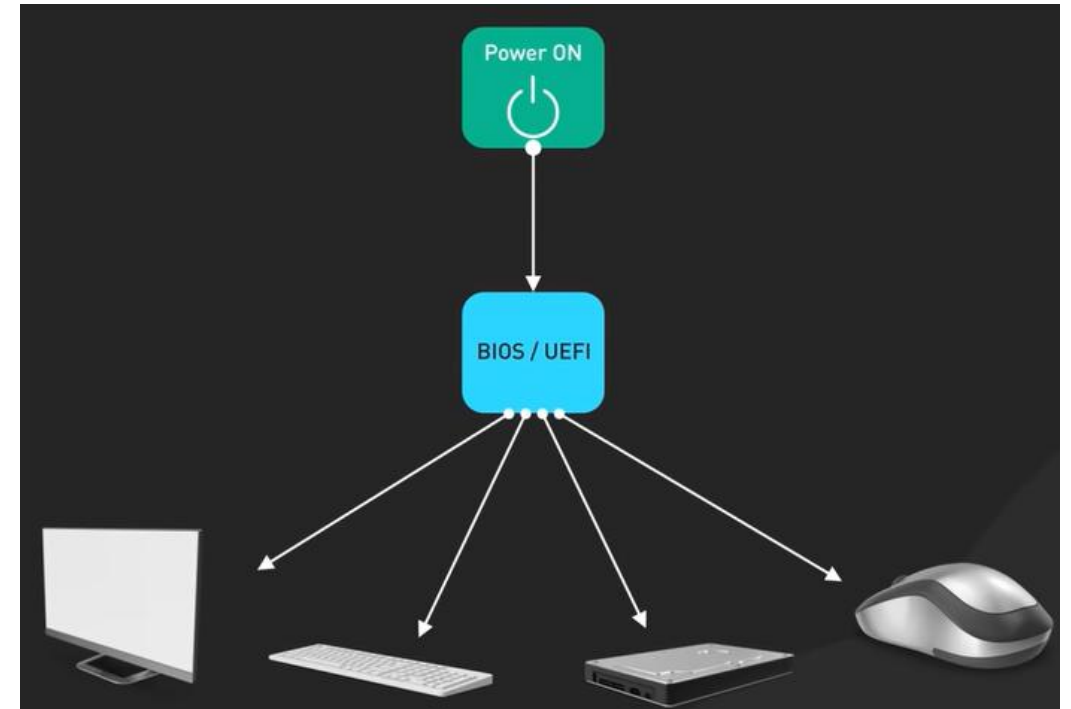
First a program called the BIOS or UEFI boots up.

These are basically pieces of software that get all the main parts of your computer ready for action: keyboard, screen, hard drive, and so on.

To access your BIOS setup screen, you can usually just press Esc, F2, F10

Basic Input/Output System vs Unified Extensible Firmware Interface:

- Boot larger drives: UEFI removes BIOS size limits and supports booting from drives up to 9.4 zettabytes.
- Faster startup: UEFI provides a more efficient, quicker boot process than traditional BIOS.
- Improved security: UEFI

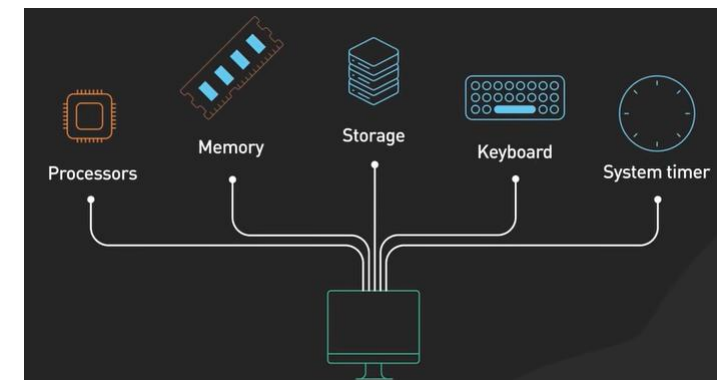
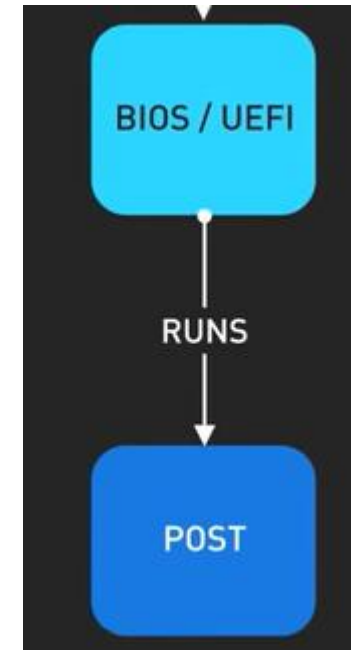


BIOS/UEFI Initialization

Next, the BIOS/UEFI runs a check called the power-on self-test (POST)

- Power-On Self-Test (POST):** basic device check on the most important hardware components which includes the RAM, CPU, and storage devices (HDD, SSD, and USB) before turning on
- To check if hardware is available for the present booting or not it will send an electric pulse to each and every device in the list that it already have. If an electrical pulse is returned from that device it will come to a conclusion the hardware is working fine and ready for use.

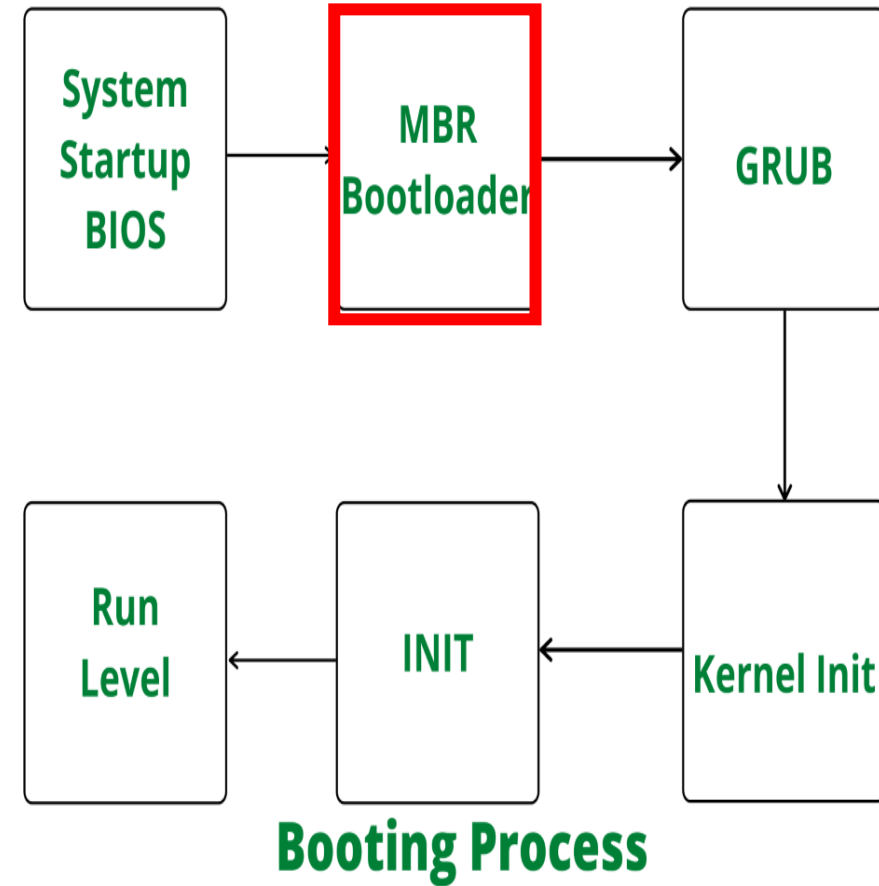
The BIOS then hands control back to the CPU to load the bootloader from that device. If the first device isn't available or doesn't contain a valid bootloader, it tries the next one in the list, then the next. If none of the devices can boot, the BIOS displays an error such as **“No boot device found.” (look hardware, cdroom..)**



MBR Bootloader

- MBR stands for **Master Boot Record** contains a **small program** that loads the full bootloader (like GRUB).
- Only 512 bytes!!
- located in the 1st sector of the bootable disk, which is typically **/dev/hda, or /dev/sda**.
- The MBR also contains information about GRUB

Its job: **find and load the next-stage bootloader** (e.g., GRUB).



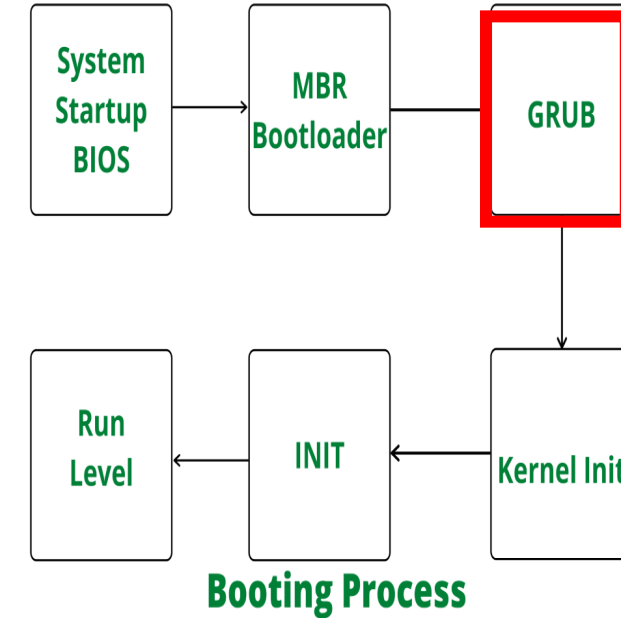
GRUB: GRand Unified Bootloader

Once the Bootloader stage 1 is completed and able to find the actual bootloader location, Stage 1 bootloader start second stage by loading Bootloader into memory

- GRUB is the most common (LILO, syslinux, etc..) and it is the first thing you see when you boot your computer. It has a simple menu where you can select some options (e.g. kernel).

```
Ubuntu 8.04, kernel 2.6.24-16-generic
Ubuntu 8.04, kernel 2.6.24-16-generic (recovery mode)
Ubuntu 8.04, memtest86+
Other operating systems:
Windows Vista/Longhorn (loader)
```

GRUB loads the user-selected (or default) kernel into memory and passes control on to the kernel. If user do not select the OS, after a defined timeout GRUB will load the default kernel in the memory for starting it

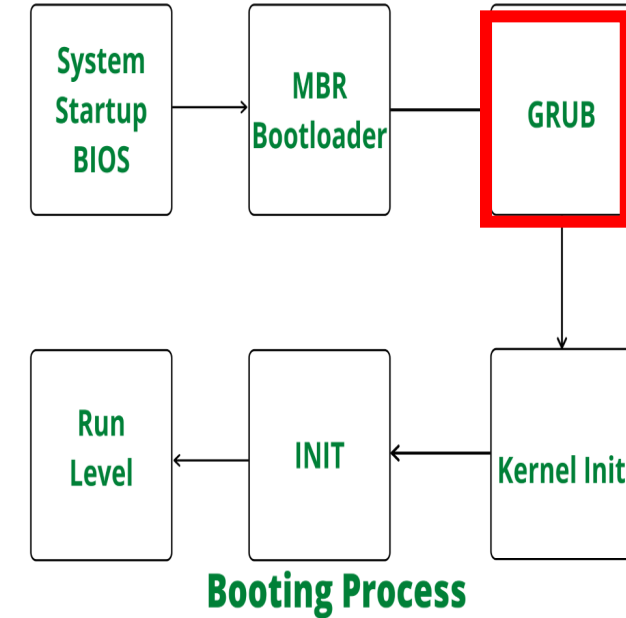


GRUB: GRand Unified Bootloader

GRUB configuration file at /boot/grub/grub.cnf or /etc/grub.conf

```
#boot=/dev/sda
default=0
timeout=5
splashimage=(hd0,0)/boot/grub/splash.xpm.gz
hiddenmenu
title CentOS (2.6.18-194.el5PAE)
    root (hd0,0)
    kernel /boot/vmlinuz-2.6.18-194.el5PAE ro root=LABEL=/
    initrd /boot/initrd-2.6.18-194.el5PAE.img
```

default=0 # Which menu entry to boot first (0 = first entry)
timeout=5 # Seconds to wait before auto-booting default entry
splashimage=(hd0,0)/boot/grub/splash.xpm.gz # GRUB background image
hiddenmenu # Hides the menu unless a key is pressed

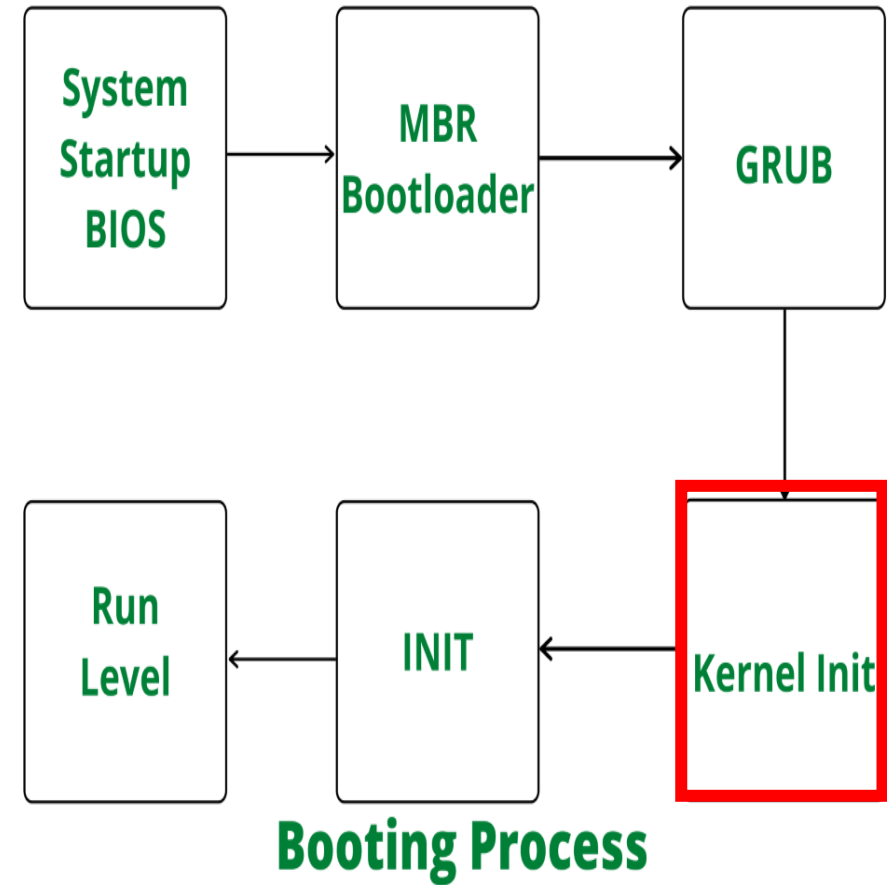


Kernel init

Once grub loads itself up it inserts the linux kernel into memory and hands control over to the kernel to finish the startup process

The kernel takes over computer's resources and starts initiating all the background processes and services

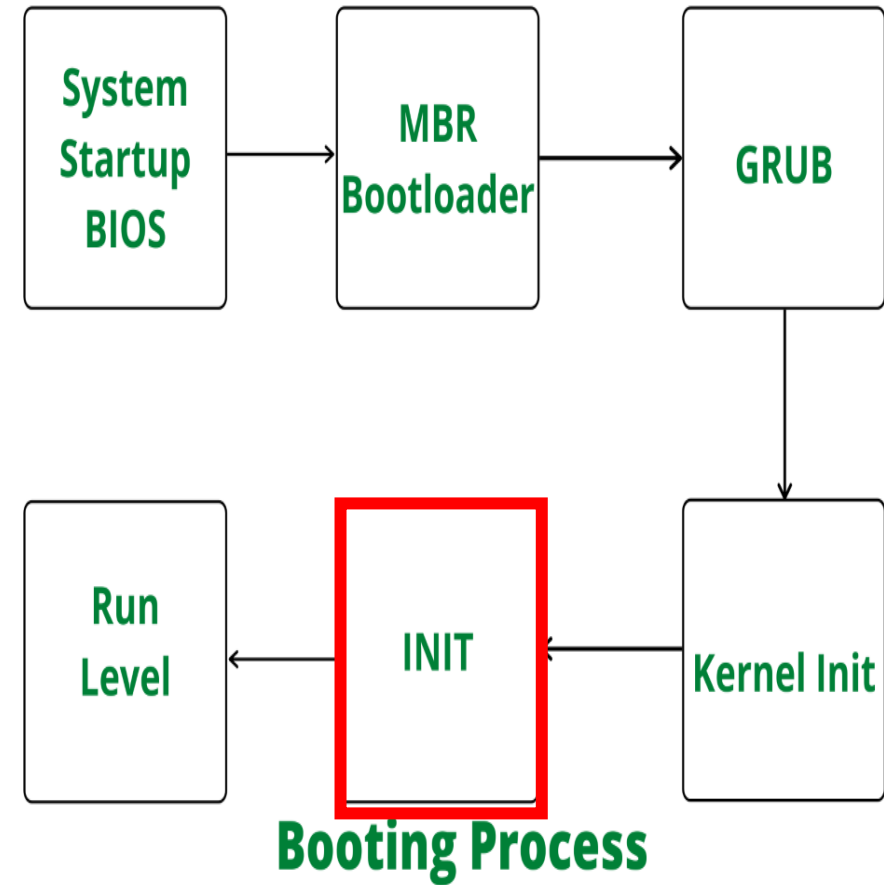
- it executes the /sbin/init program, which is always the first program to be executed. You can confirm this with its process id (PID), which should always be 1 (ps -ef | grep init)



INIT

init system:

- It is the first user-space process started by the kernel after system initialization.
 - It always has PID 1, making it the ancestor of all other user-space processes.
 - Its job is to bring the system from boot to a fully usable state.
 - On modern Linux systems, init is usually replaced by systemd
-
- Starts essential services and daemons like sshd (remote login), cron (scheduled tasks), and network services (DHCP, DNS).
 - Mounts file systems such as /home and /var, ensuring all partitions are accessible before services start.
 - Initializes system logging with tools like rsyslog or journald to capture events and errors.
 - Configures networking, bringing up interfaces and assigning IP/DNS settings.
 - Enters the default system state (runlevel or systemd target) and starts login managers (getty, GDM), enabling user access.



Run Level

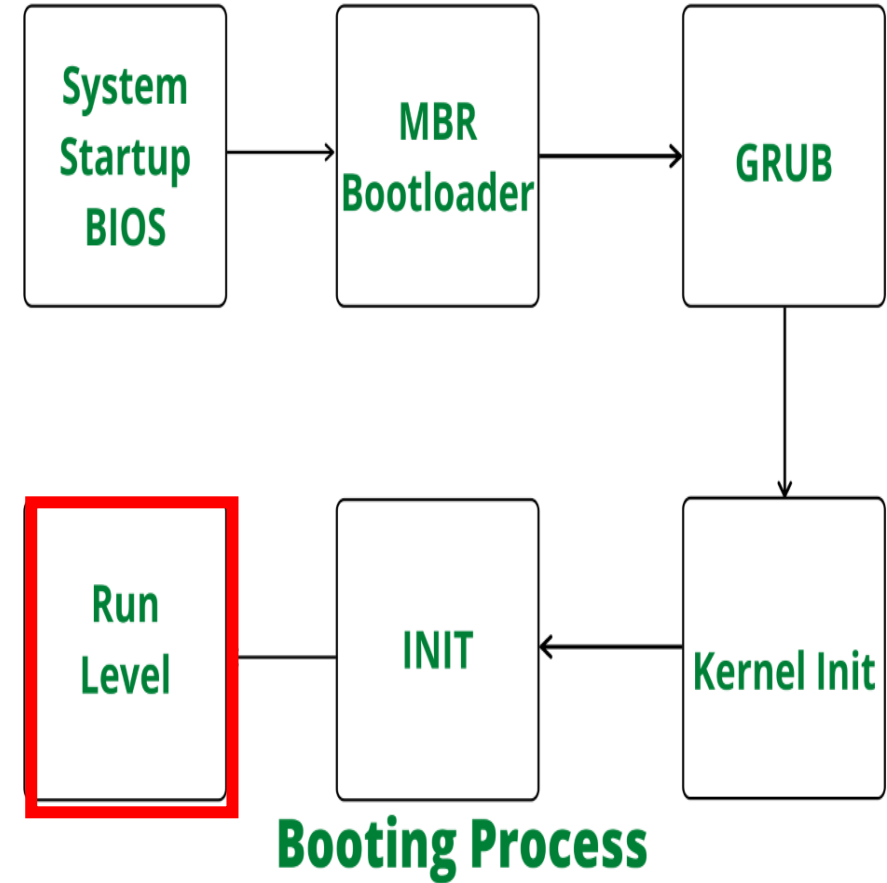
A runlevel defines the state of the system and which services are running.

Examples:

- 0: Halt
- 1: Single-user (rescue)
- 3: Multi-user (no GUI)
- 5: Multi-user with GUI
- 6: Reboot

With systemd, these are replaced by targets (e.g., multi-user.target, graphical.target).

- 0 → poweroff.target
- 1 → rescue.target (Minimal services, mostly for maintenance or recovery. No networking)
- 3 → multi-user.target (Full multi-user mode without GUI)
- 5 → graphical.target (Full multi-user with GUI)
- 6 → reboot.target
- Emergency → emergency.target



Linux file system structure

What is a file system?

Linux file system structure

What is a file system?

- Organizes data on disk
- Stores files and directories
- Defines access permissions

Unix File System is a logical method of organizing and storing large amounts of information in a way that makes it easy to manage

Linux file system structure

Popular File Systems

- **FAT** (File Allocation Table)
 - Old file system used by MS-DOS, early Windows
- **NTFS** (New Technology File System)
 - Modern Windows file system (permissions, encryption)
- **ext (ext2/ext3/ext4)** (Extended Filesystem)
 - Common file system for Linux
- **HFS+** (Hierarchical File System Plus)
 - Older macOS file system
- **APFS** (Apple File System)
 - New file system for modern macOS and iOS

Linux file system structure

What is a file?

Linux file system structure

What is a file?

- **Ordinary files (-):** text, data, or program information. (avoid space or special character in name)
- **Directories (d):** containers or folders that hold files, and other directories. They do not contain files, only entries that point to them.
- **Devices:** In a Windows-like operating system, devices like CD-ROM and hard drives are represented as drive letters like F: G: H, whereas in the Linux system, devices are represented as files. As for example, /dev/sda1, /dev/sda2, and so on.
- **Links (l):** Shortcuts or references to files (hard/soft). A **hard link** is a second name for a file. A **symbolic (soft) link** is more like a shortcut pointing to another file or folder.

EVERYTHING IS A FILE

(even directories, devices, and links)

Linux file system structure

File convention?

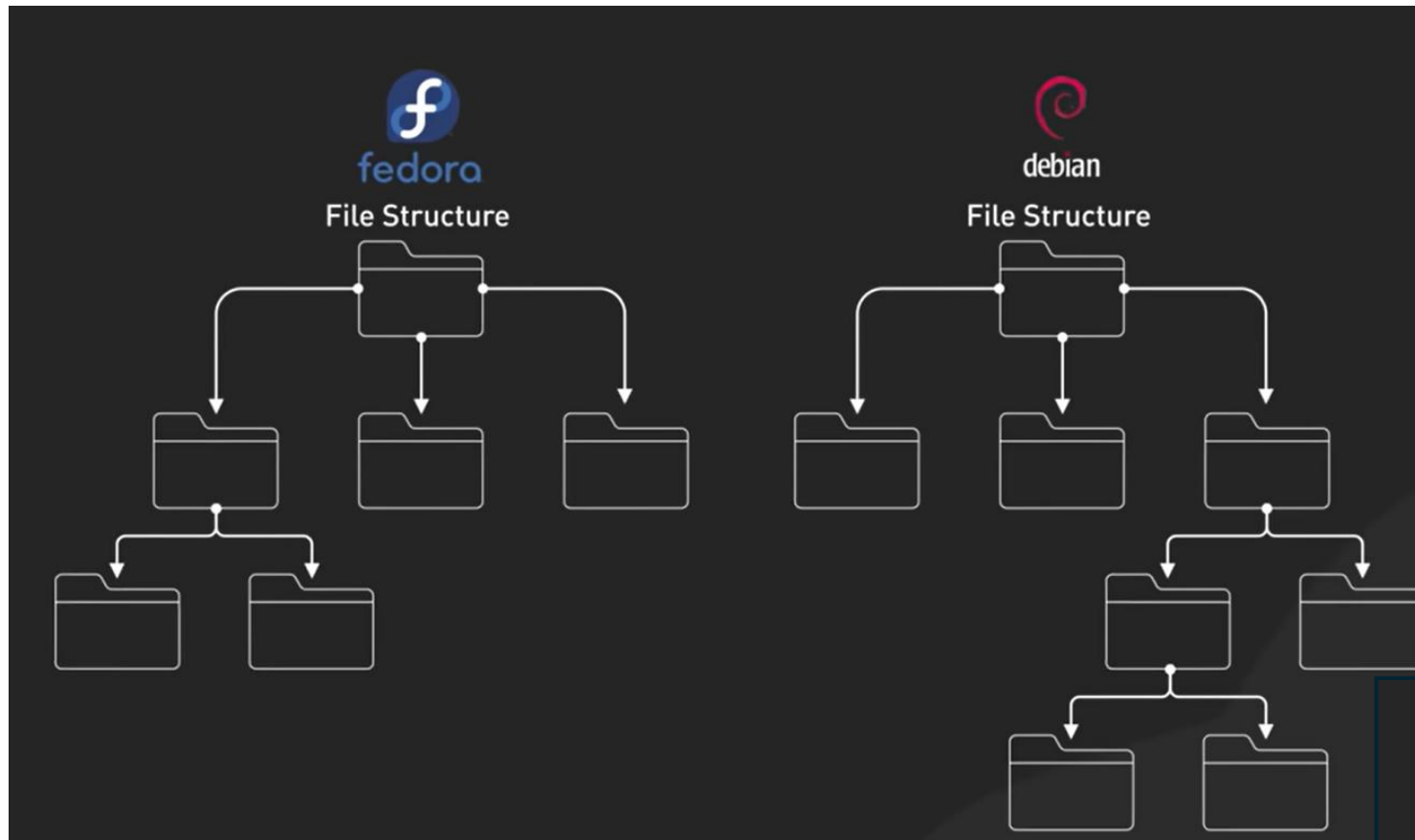
- Filenames are **case-sensitive**. For example, '**File.txt**' and '**file.txt**' are considered two distinct files
- Special characters like `/`, `*`, `?`, `:`, `$`, `%`, are not typically allowed in filenames.

Directory structure

How does linux organize the folders?

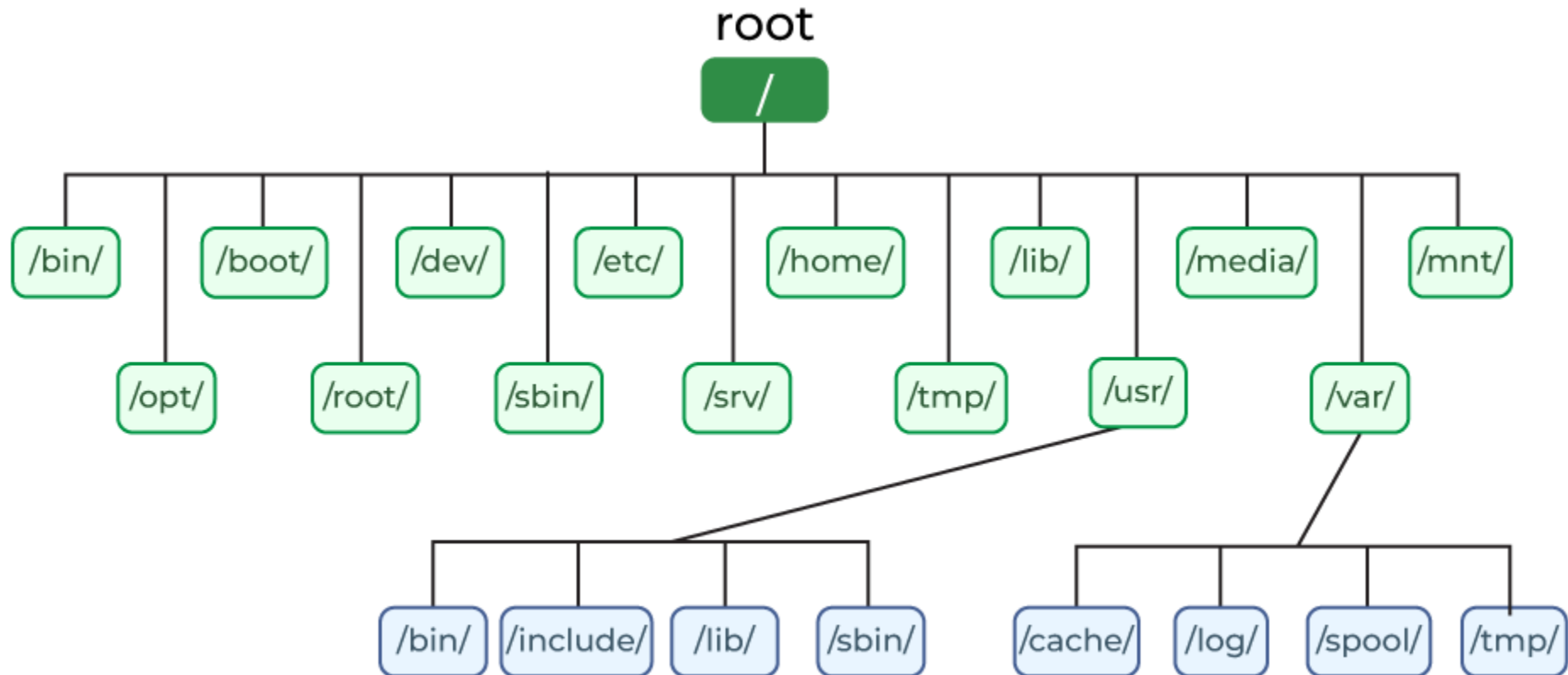
Directory structure

At the beginning, each distribution has its own organization!!



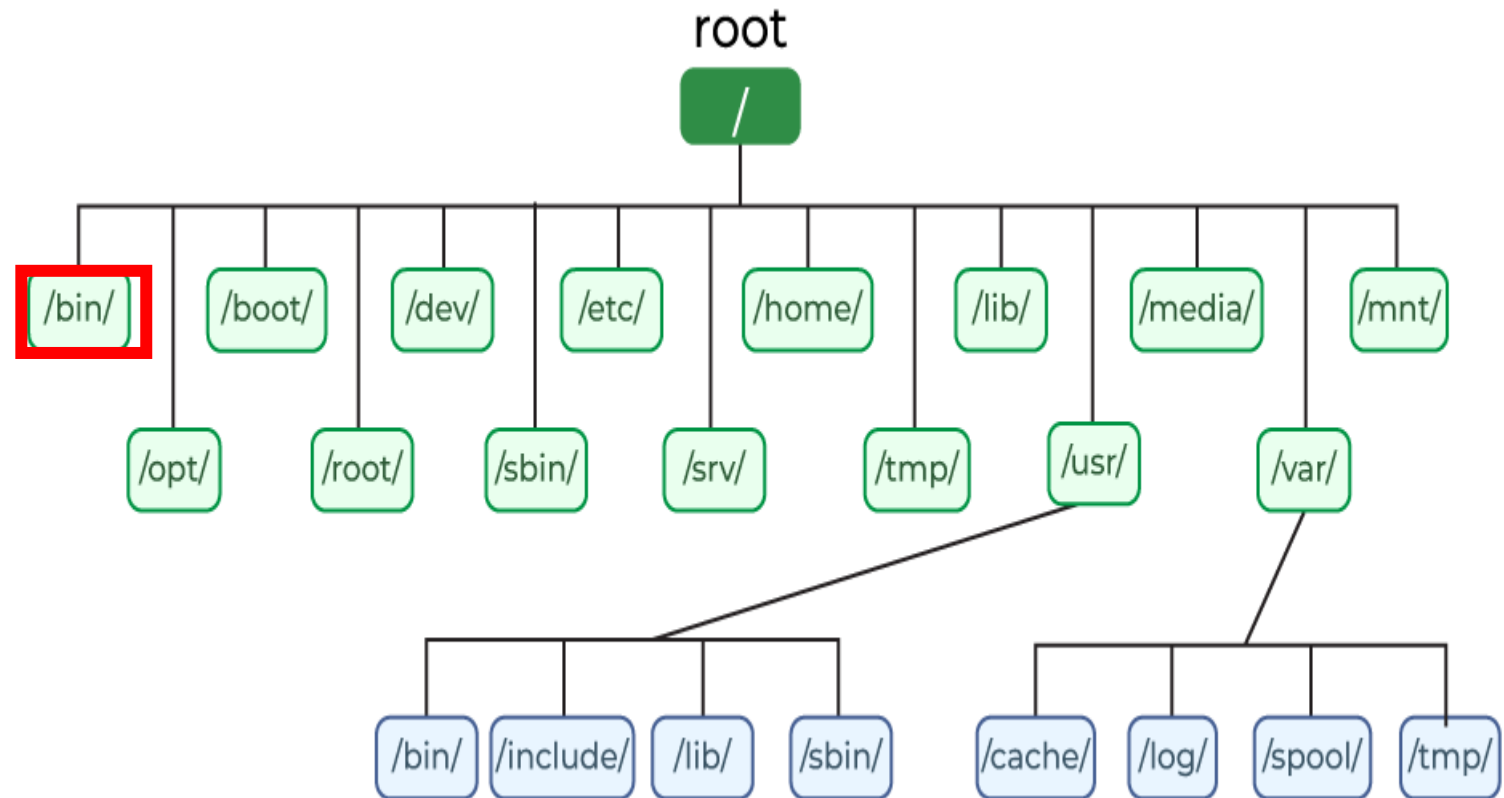
Directory structure

hierarchical tree structure which is anchored at a special top-level directory known as the root (designated by a slash '/').



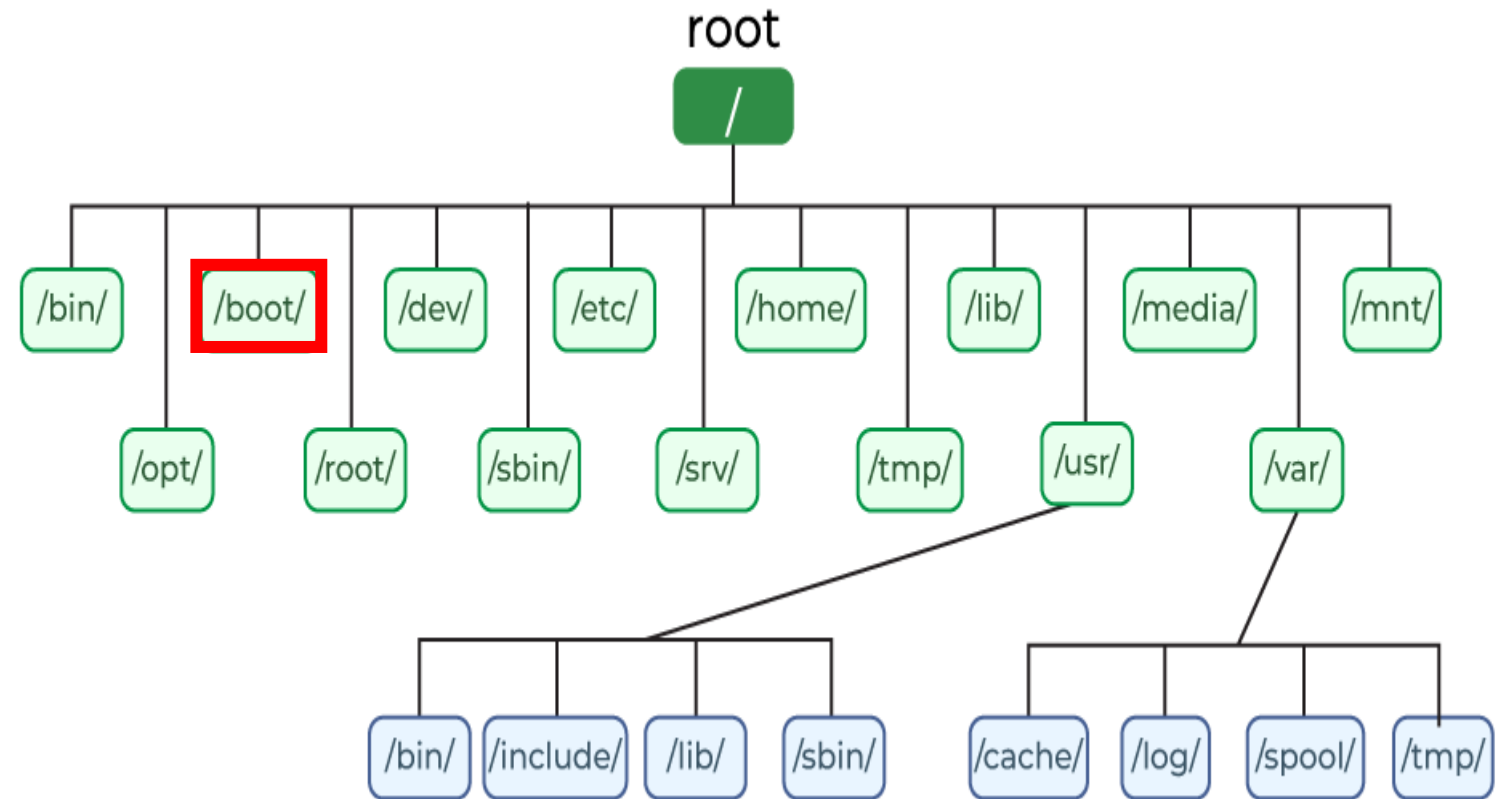
Directory structure

- /bin: Essential command binaries that need to be available in single user mode; for all users.
- Holds simple command-line tools you use every day cat, ls, cp.



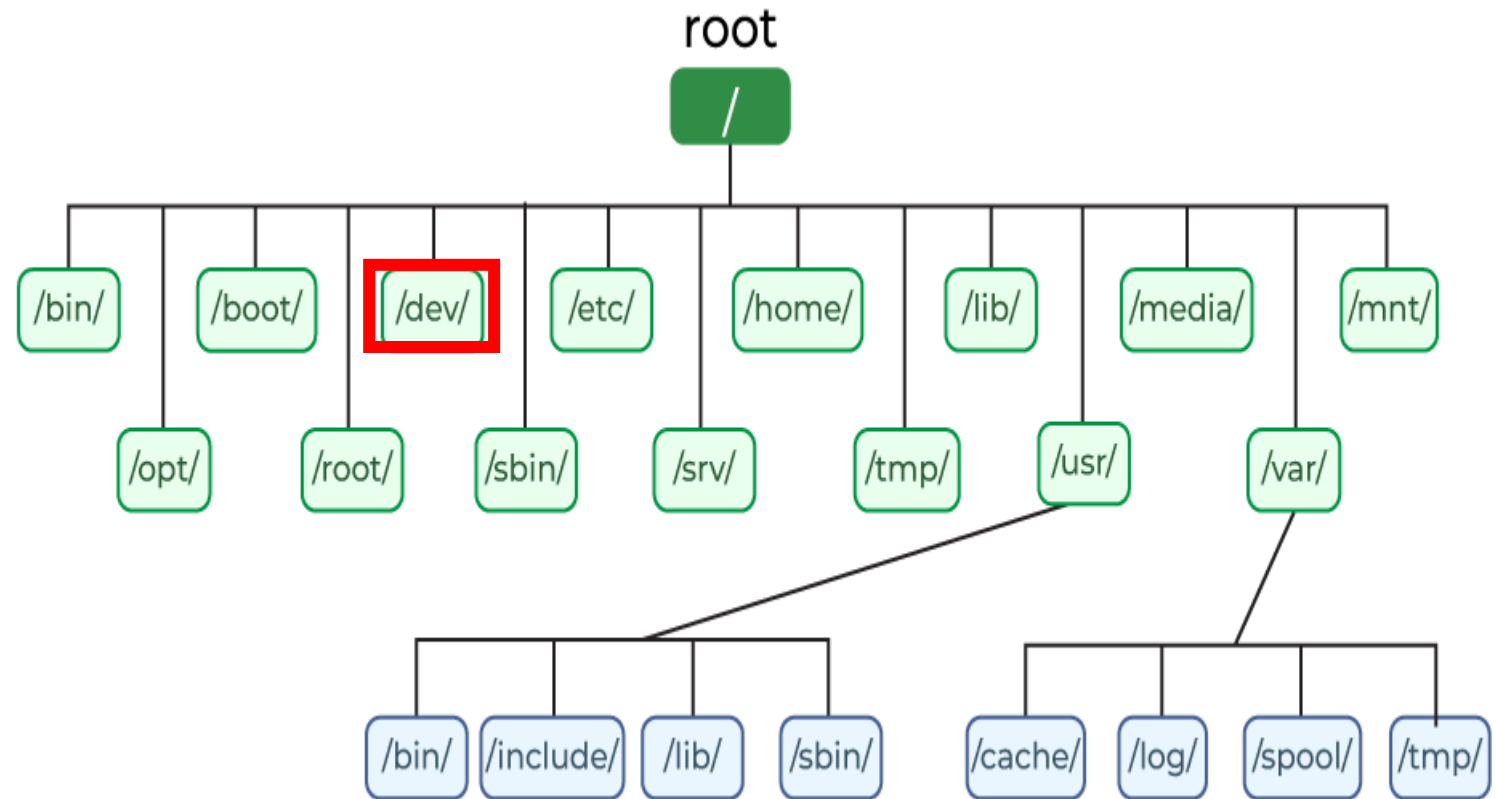
Directory structure

- /boot: Boot loader files, e.g., kernels, initrd.



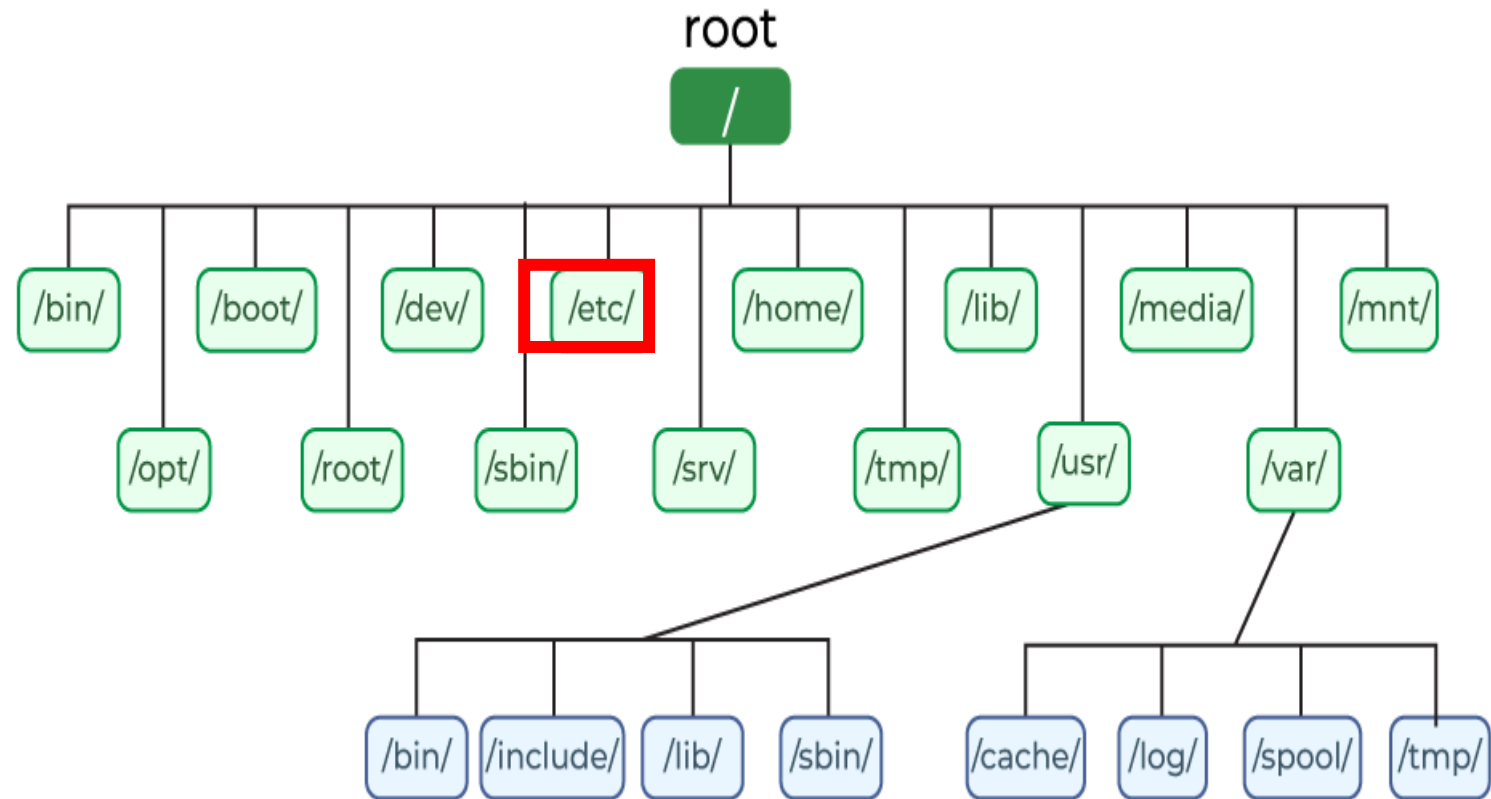
Directory structure

- /dev: system device files
- Represents hardware devices as files (e.g., /dev/sda for disks, /dev/null, /dev/tty).
- Used to interact with hardware through file operations



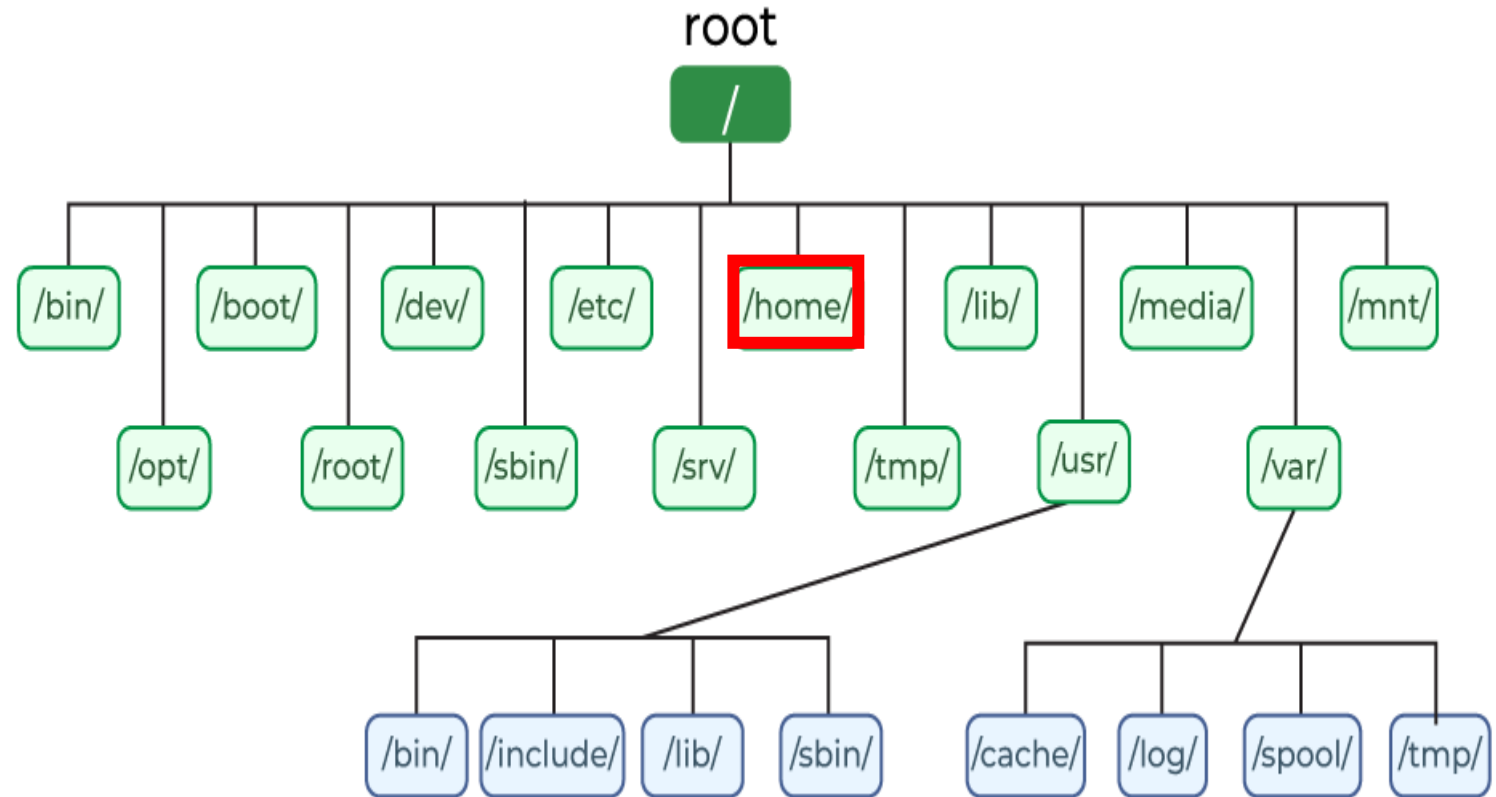
Directory structure

- `/etc`: Stores system-wide configuration files (e.g., `/etc/passwd`, `/etc/hostname`, `/etc/hosts`).
 - Editable by admins to configure the system and services.
-
- `/etc/bashrc` System-wide defaults and aliases for the Bash shell.
 - `/etc/crontab` Shell script that schedules commands to run at specific intervals.
 - `/etc/exports` Defines filesystems available to the network (NFS).
 - `/etc/fstab` Lists disk drives and their mount points.
 - `/etc/passwd` User account information (public part).



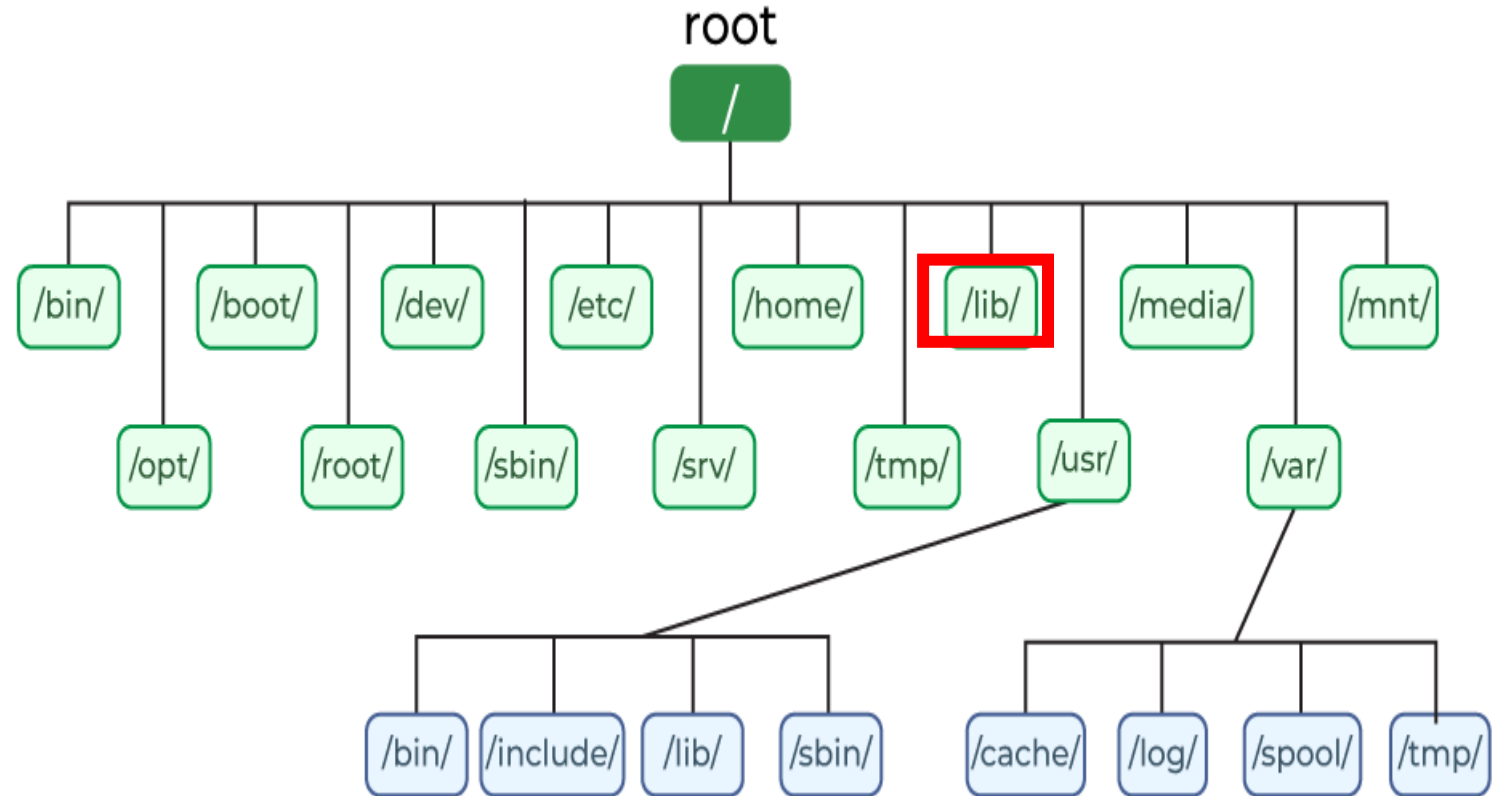
Directory structure

- /home: Contains the home directories for the users.
- Stores user data, configuration files, downloads, etc.



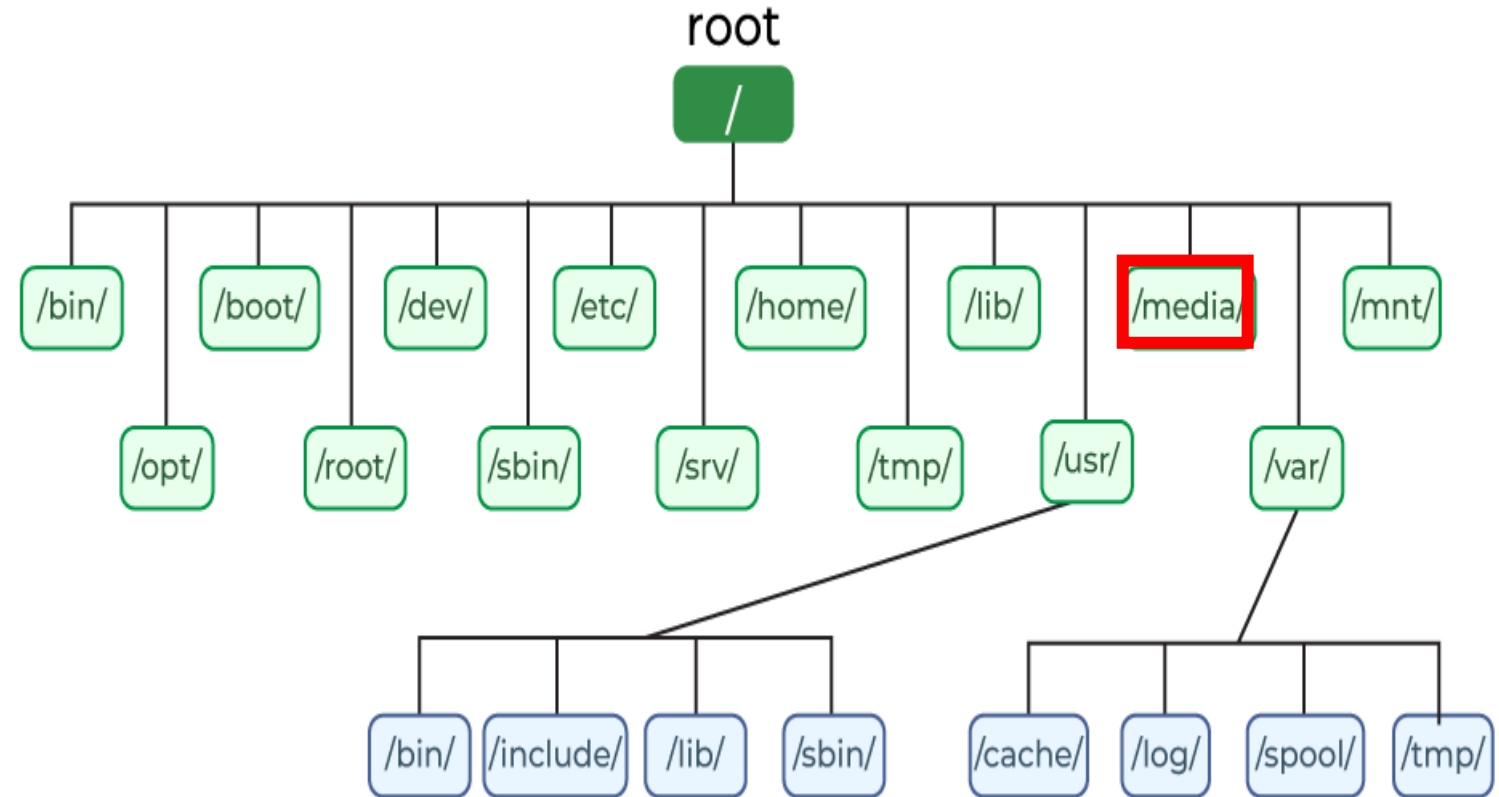
Directory structure

- /lib: Contains system libraries, and some critical files such as kernel modules or device drivers.
- Contains libraries needed by binaries in /bin and /sbin.



Directory structure

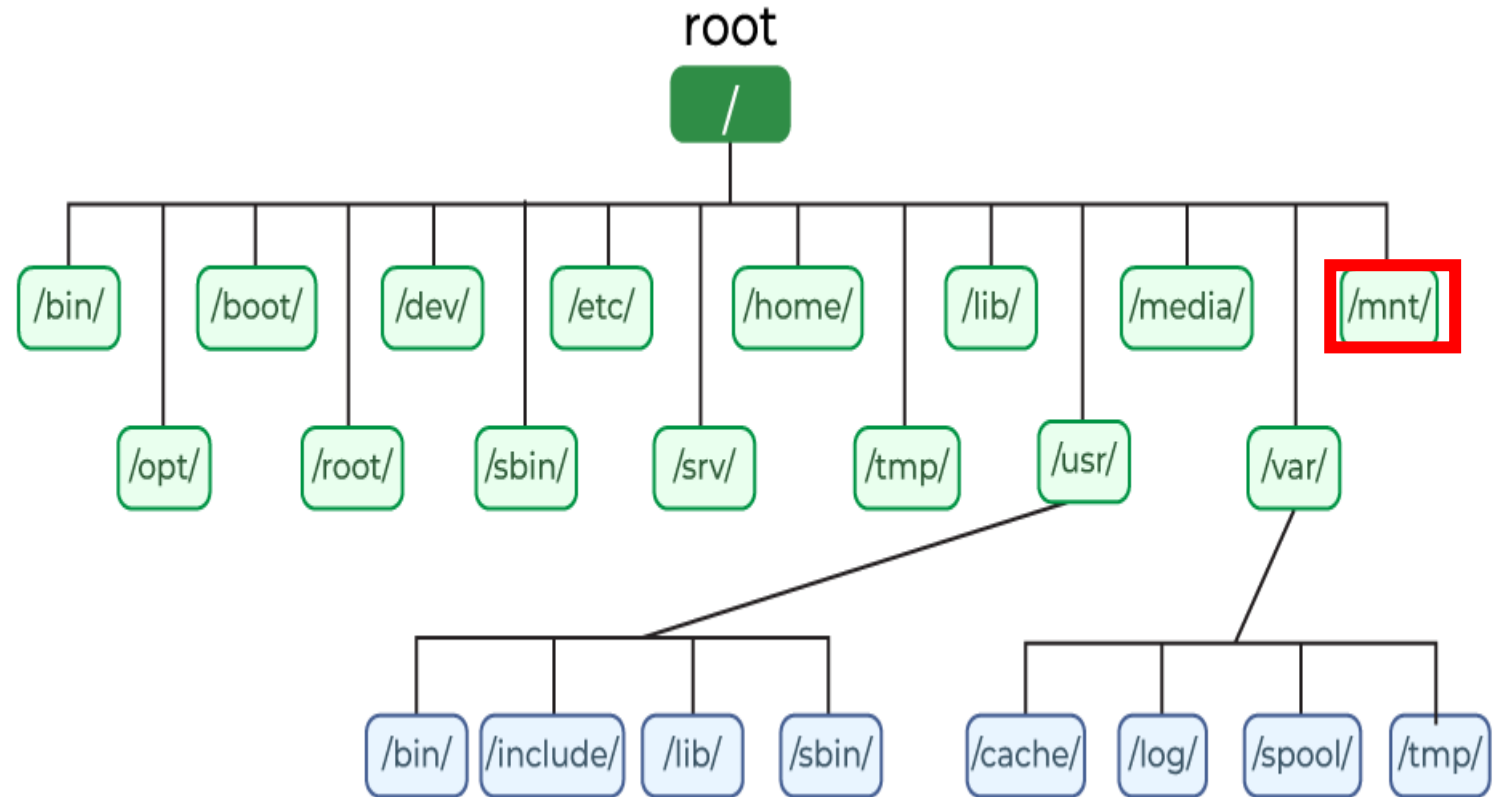
- /media: For automounted devices (e.g., USB sticks, CDs).



Directory structure

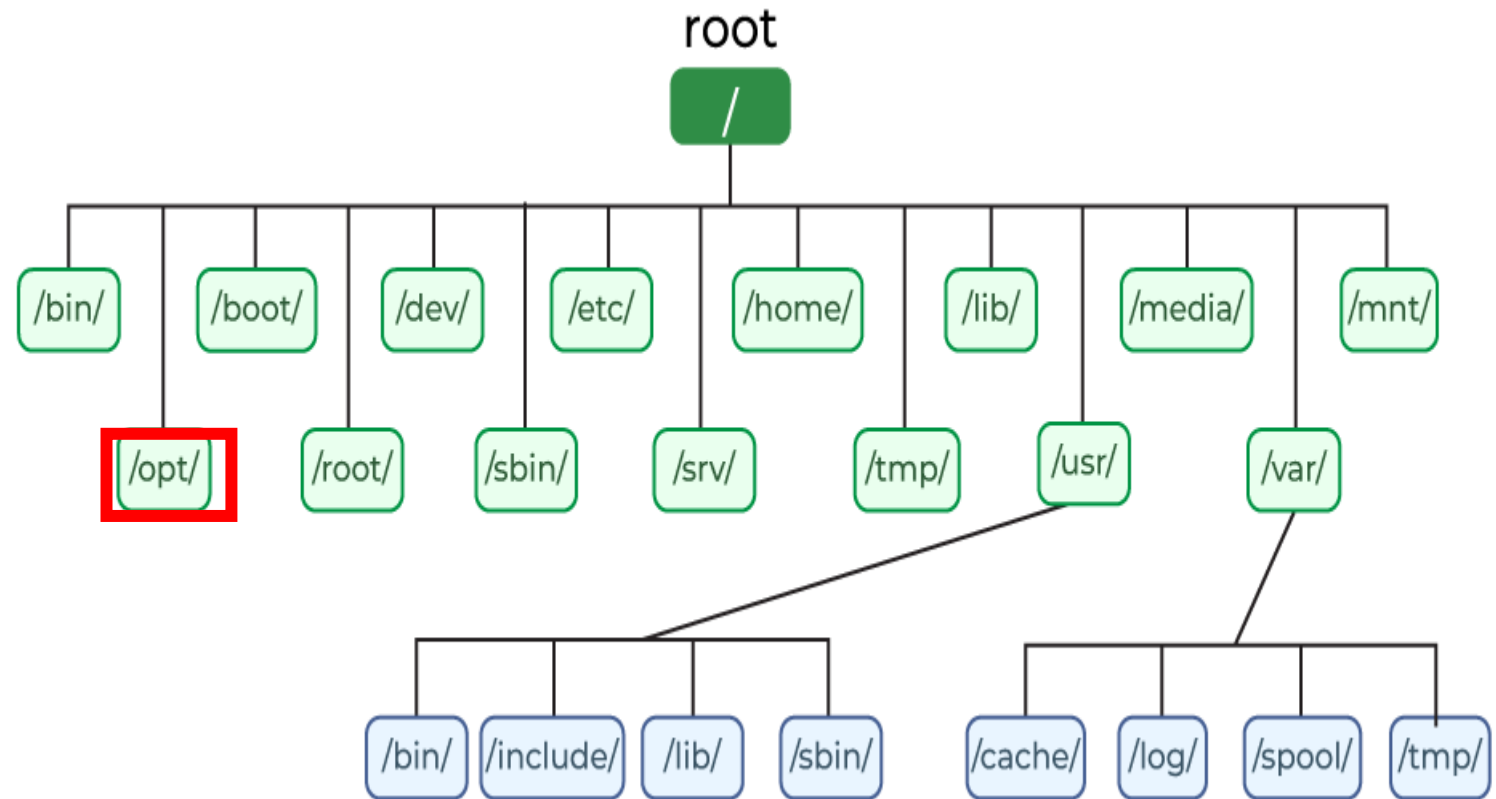
- /mnt: “mount” Contains filesystem mount points.
- For temporarily mounting filesystems manually

These are used, for example, if the system uses multiple hard disks or hard disk partitions. It is also often used for remote (network) filesystems, CD-ROM/DVD drives, and so on.



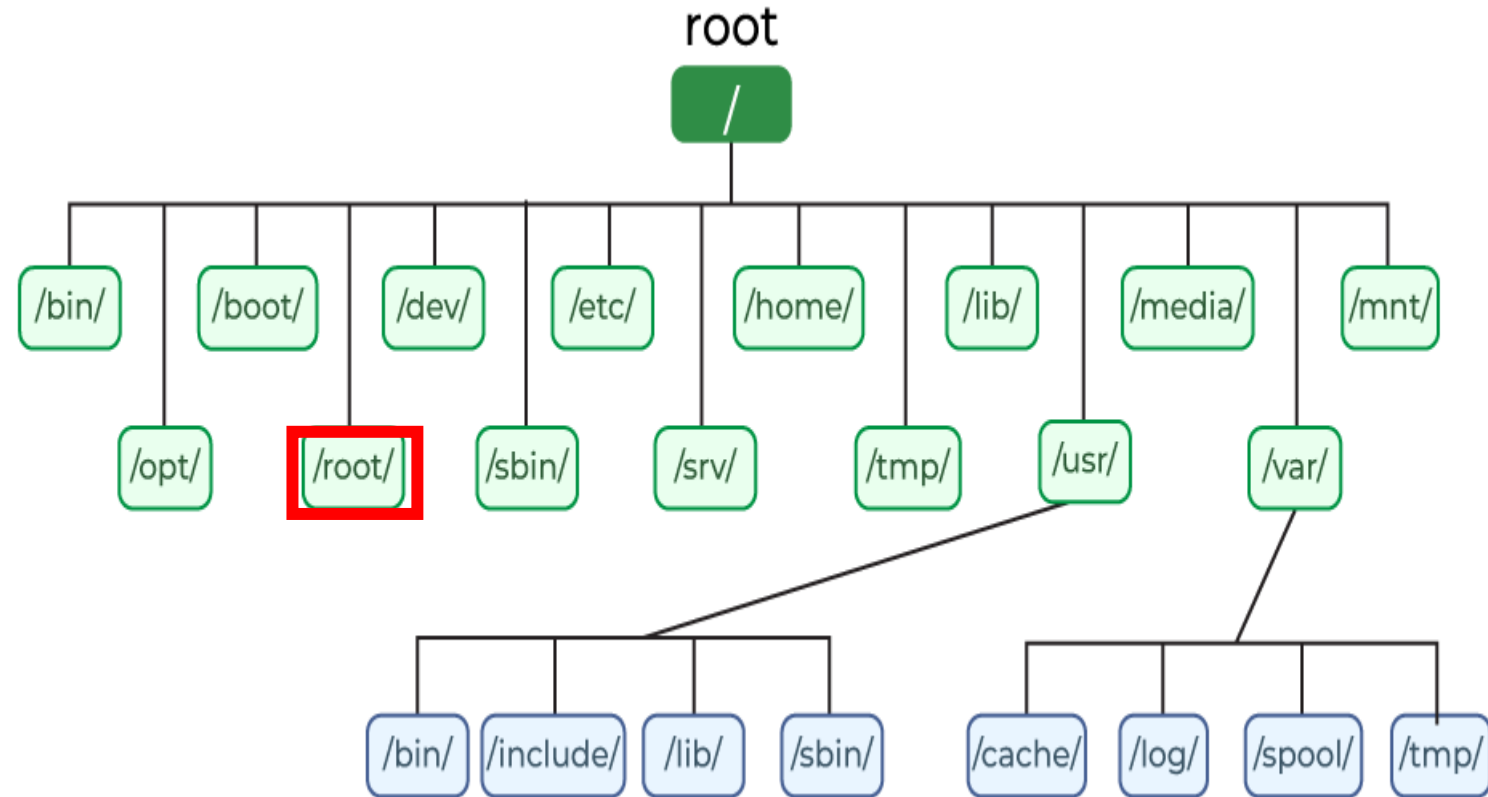
Directory structure

- /opt: optional application software packages that are not part of the standard Linux distribution.
- For large third-party apps that aren't part of the system.



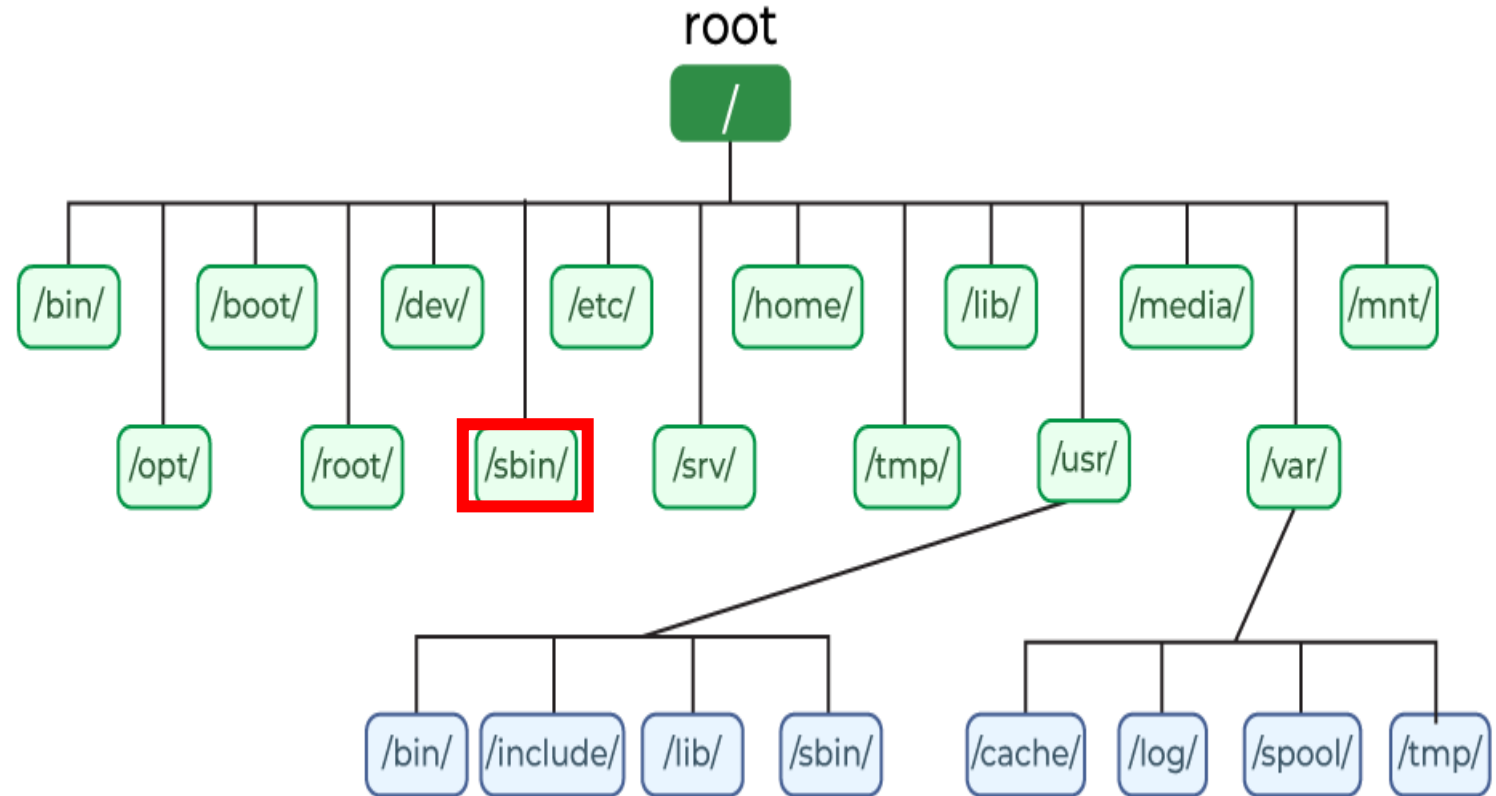
Directory structure

- /root: Home directory for the root user.
- Always available even if /home/ fails



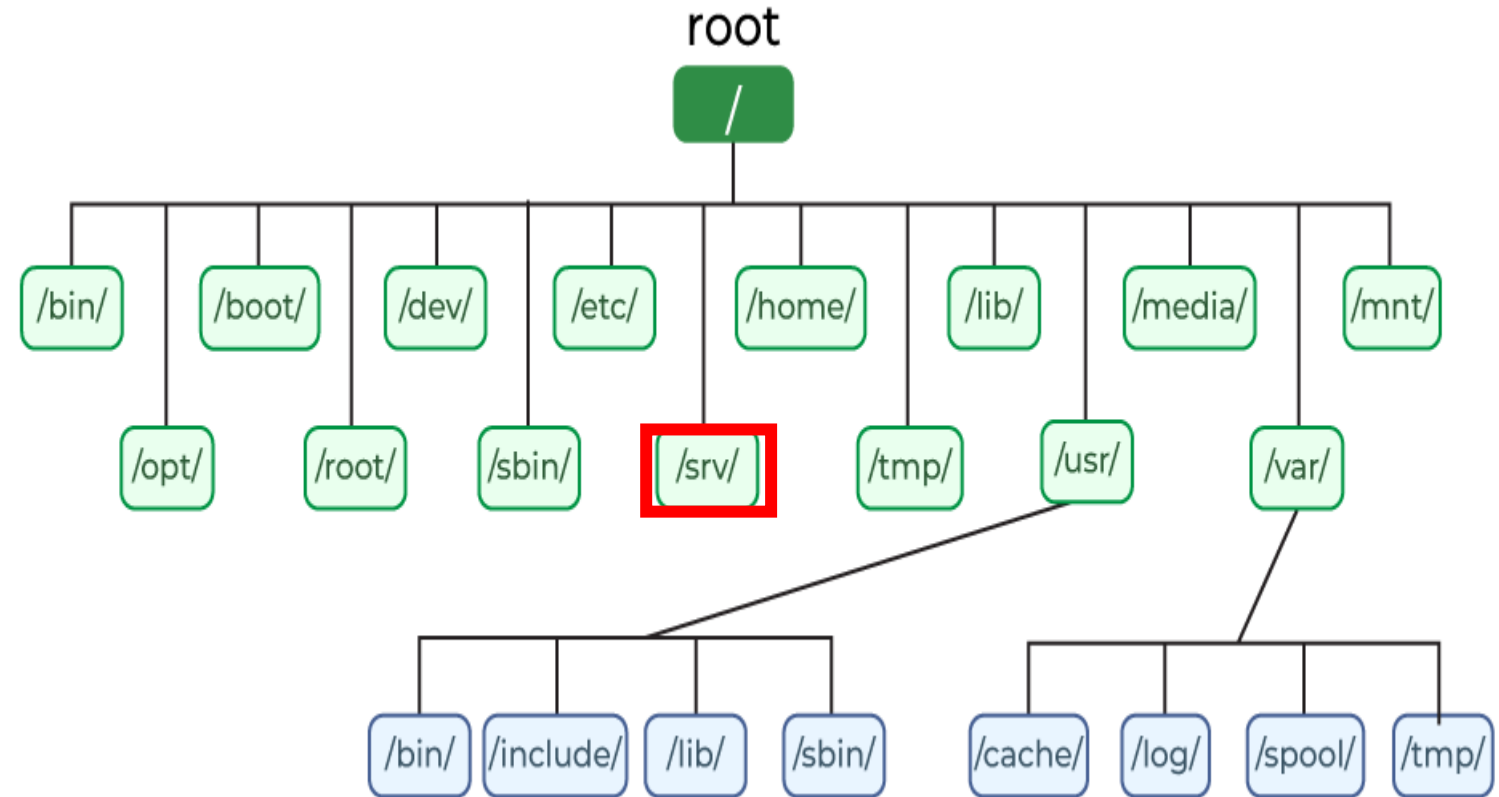
Directory structure

- /sbin: Superuser system utilities (for performing system administration tasks).
- Contains binaries for system administration (e.g., reboot, shutdown, fsck).
- Used by root or scripts during system boot and maintenance



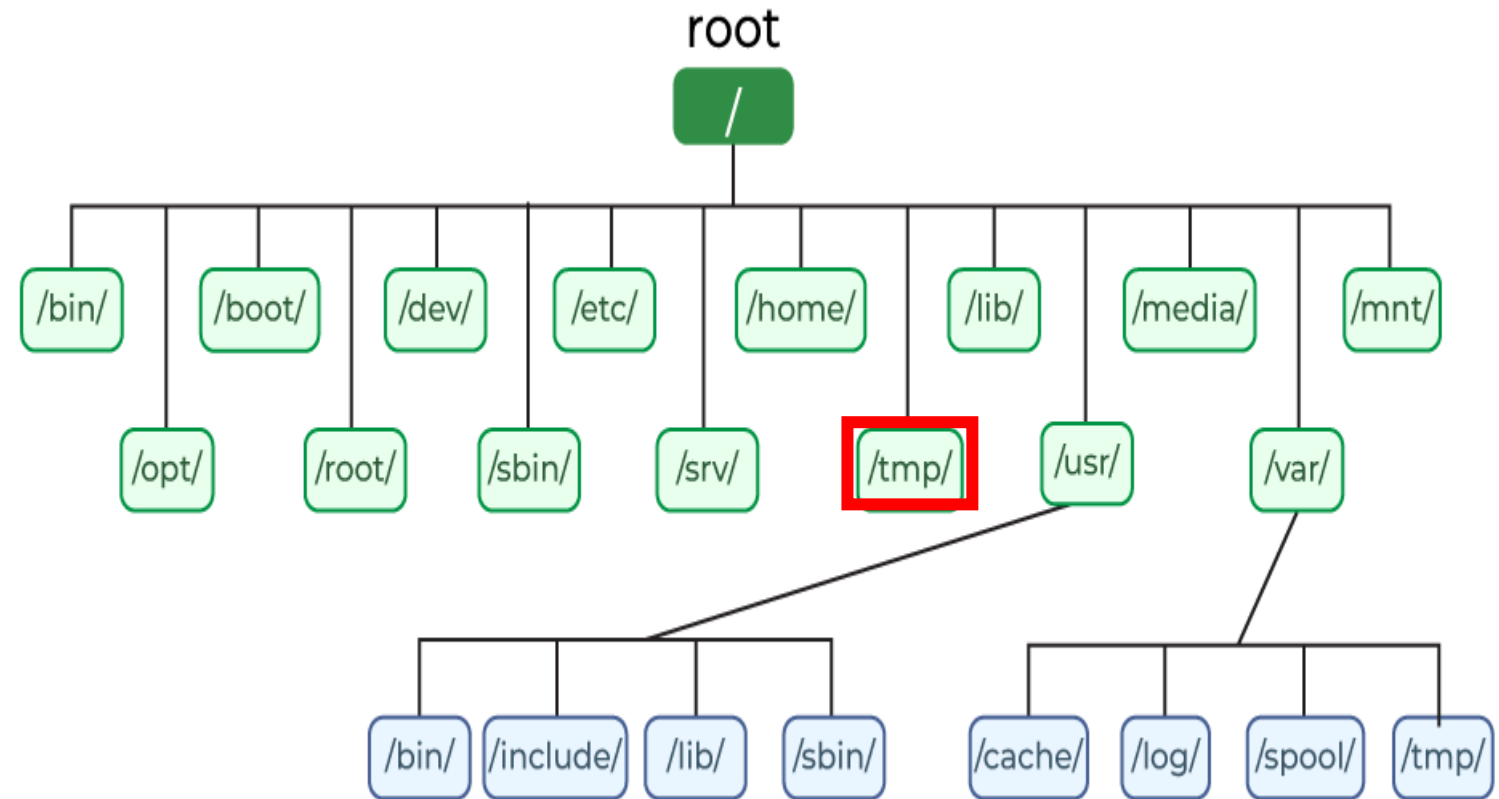
Directory structure

- /srv: “service” Contains **data served by system services** such as web servers, FTP servers, or databases.



Directory structure

- /tmp: temporary files.

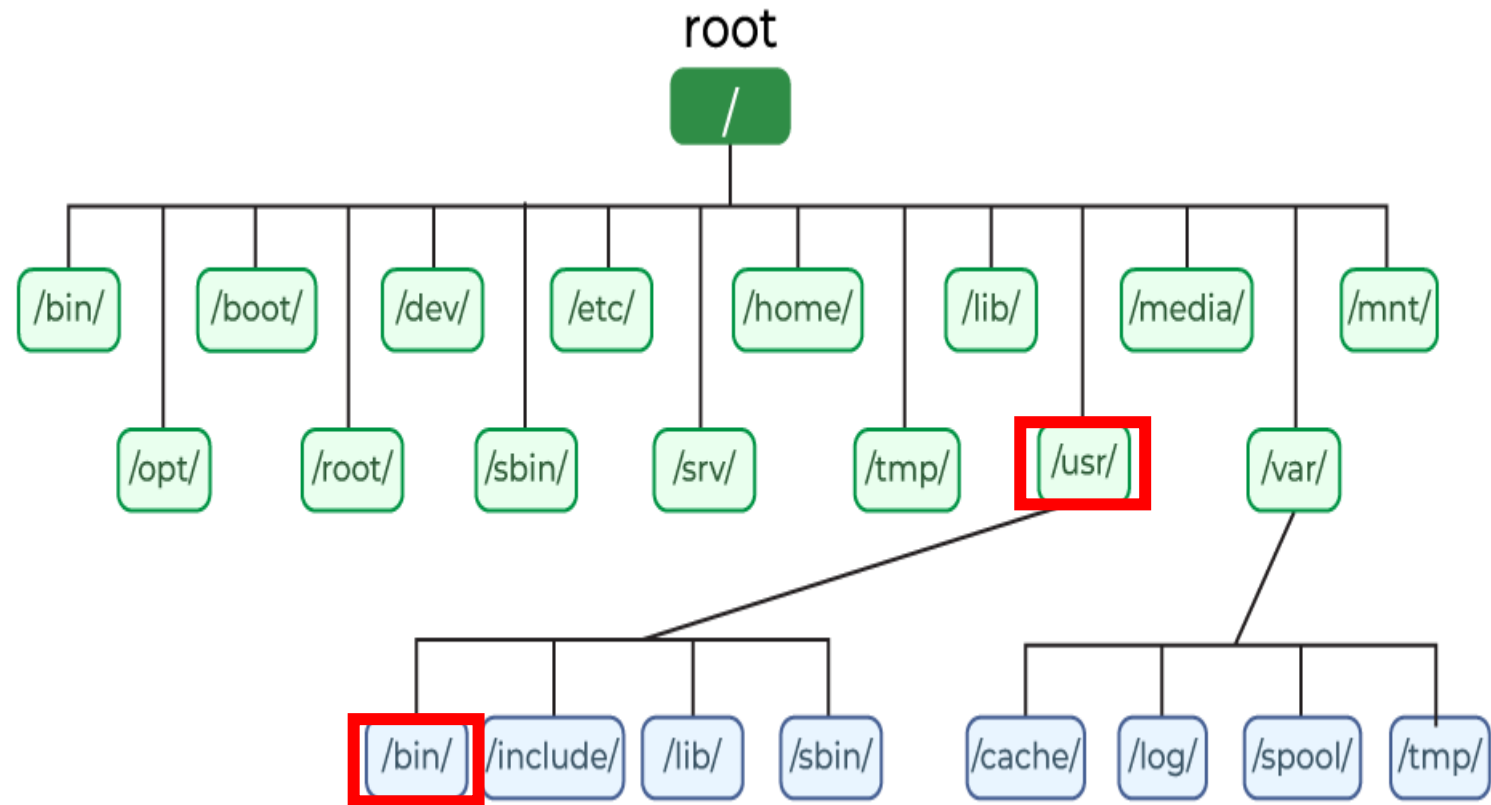


Directory structure

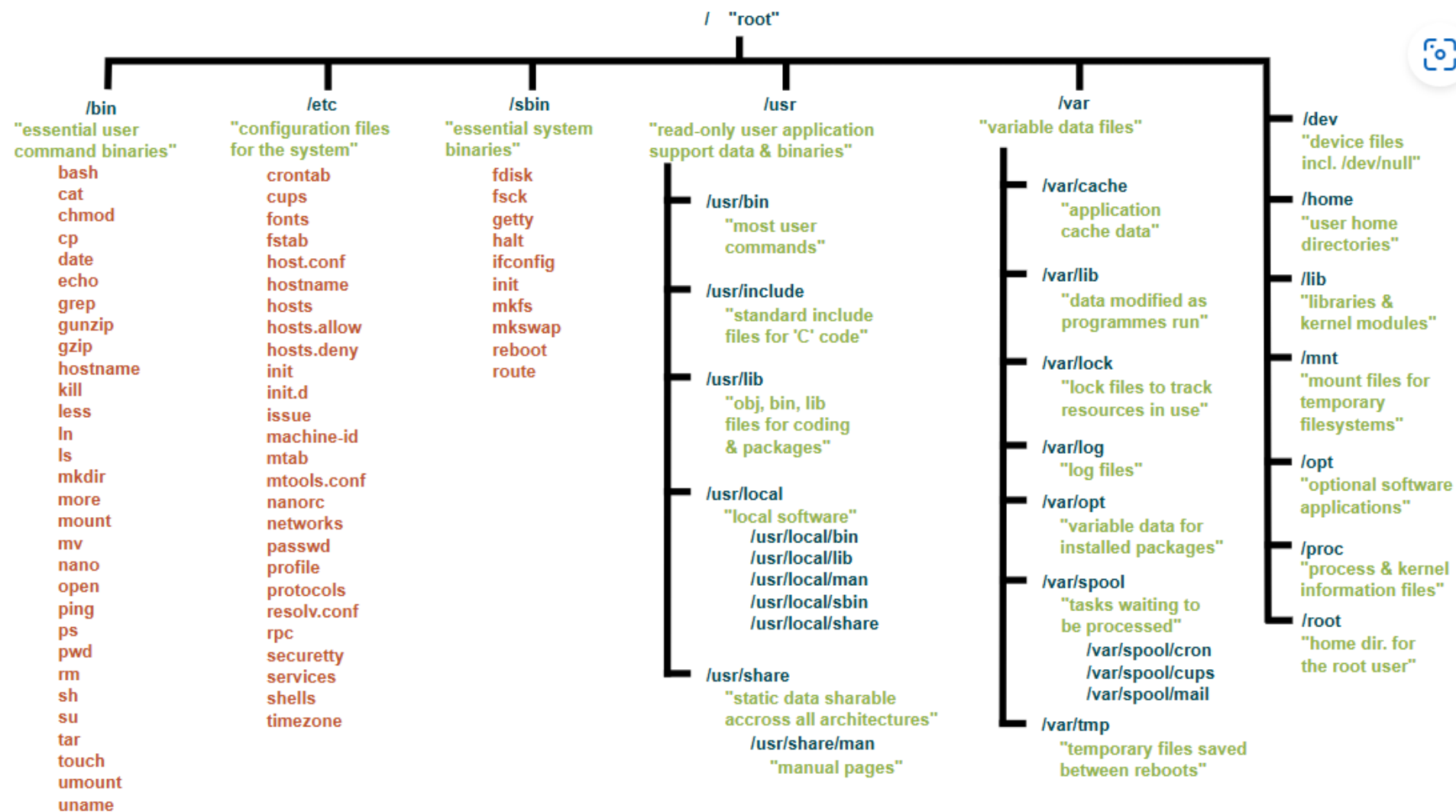
- `/usr/bin`: This directory stores all binary programs distributed with the operating system not residing in `/bin`, `/sbin` or (rarely) `/etc`

eg.: `git`, `wget`

- `/usr/bin` Contains most user-level executable programs.
- `/usr/bin/X11` Symbolic link to `/usr/bin`, historically used for X11 executables.
- `/usr/include` Header files for C programs (standard library headers, system headers).
- `/usr/share` Architecture-independent shared files (docs, icons, locale, man pages).
- `/usr/lib` Libraries and object files required by programs in `/usr/bin`.
- `/usr/sbin` System administration commands (usually run by root).



Directory structure



Paths

What is a path?

- A path is a unique location to a file or folder in the filesystem.
- Paths use / and alphanumeric characters to describe locations

Paths

What is a path?

- A path is a unique location to a file or folder in the filesystem.
- Paths use / and alphanumeric characters to describe locations

Two types of paths:

- **Absolute Path:** starts from the **root directory** / and specifies the full path to a file or folder, no matter what your current directory is
- **Relative Path:** Path relative to your current directory

Basic commands

Commands to use:

- `pwd` → Show where you are (Print Working Directory)
- `ls` → List files and directories
- `cd` → Change directory

Basic commands

Quick Tasks:

- Move to the root directory:

```
$ cd /
```

- List what's inside /home/:

```
$ cd /home
```

```
ls
```

- Find what's inside /etc/:

```
cd /etc
```

```
ls
```

- Come back to home directory:

```
cd ~
```

Basic commands

Exercises :

As you, I am an astronomer. Do you know what is a black hole? They're huge concentrations of matter packed into very tiny spaces. A black hole is so dense that is strong enough that nothing – not even light – can escape.

Do you know that there is a black hole in Linux? You will explore the directories and find it.

- Start looking in the folder that provides access to peripheral devices (like disks, keyboards, and other “hardware”).
- Look for a device whose name suggests nothingness.
- Check what happens when you redirect output to it. (echo "Hello" >.....)
- What is the purpose of this black hole?

Basic commands

Exercices :

As you, I am an astronomer. Do you know what is a black hole? They're huge concentrations of matter packed into very tiny spaces. A black hole is so dense that is strong enough that nothing – not even light – can escape.

Do you know that there is a black hole in Linux? You will explore the directories and find it.

- Start looking in the folder that provides access to peripheral devices (like disks, keyboards, and other “hardware”).
- Look for a device whose name suggests nothingness.
- What is the purpose of this black hole?
- Answer: /dev/null, discarding output, Testing Command (dummy input or output file for commands that require a file but where you don't want to provide actual data).

Basic commands

Exercises :

- Continue to explore the filesystem tree using `cd`, `ls`, `pwd`. Look in `/bin`, `/usr/bin`, `/sbin`, `/tmp` and `/boot`. What do you see? What are the permissions on `/etc/passwd`
- Run `echo $SHELL`, `echo $0`, and `ps -p $$`.
- Explore `/dev/`: Identify your terminal device.
- Use `who am i` and `tty`.
- Can you find your USB or disk partitions?
- Inspect `/boot/` : what kernel version is installed? Cross-check your user looking into `proc/`
- Open `/etc/fstab` : what filesystems are auto-mounted?
- Use `systemctl list-units` to view `systemd` targets.
- Why `cd ../` does not give an error message
- Give 2 different ways to go to your home directory

Basic commands

More Tasks (sol):

- `/bin` Essential system programs like `ls`, `cp`, `mkdir`, `bash`
- `/usr/bin` Most user applications like `python3`, `gcc`, `vim`, `nano`
- `/sbin` System administration programs like `ifconfig`, `reboot`, `fdisk`
- `/tmp` Temporary files (might be empty if recently rebooted)
- `/boot` Boot files, including `vmlinuz` (kernel image) and `grub` configuration
- `echo $SHELL` → e.g., `/bin/bash`; `echo $0` → current shell.
- `ps -p $$` → shows the shell process info.
- `tty` or `who am i` → shows current terminal device (e.g., `/dev/pts/0`).
- Use `lsblk` or `df -h` to list block devices and mounted partitions.
- Character devices `/dev/tty0`, `/dev/tty1`, `/dev/console`, `/dev/random`
- Block devices `/dev/sda`, `/dev/sdb`, `/dev/loop0` (disks and partitions)
- `ls /boot/` → look for files like `vmlinuz-5.15.0-XX` for kernel version.
- `cat /etc/fstab` → shows device mount rules, e.g., `/dev/sda1 / ext4 defaults 0 1`.
- `systemctl list-units --type=target` → shows active systemd targets.
- Because `../` exists and it points back to `/` itself.
- `cd` , `cd ~`, `cd $home`